

PREVAIL — Master Engineering Plan (Definitive)

Version: 2.0 (post-discussion source of truth)

Date: 23 September 2026

Status: Authoritative implementation roadmap — no code in this document

Audience: Implementation engineers, research reviewers, demo stakeholders

Document control

This plan supersedes the initial engineering plan (v1.0). It incorporates:

- Original PREVAIL research architecture (predict → pre-position → warm shadow → promote).
 - Prototype stack: Rust runtime, Apache Flink workload, Python/PyTorch GRU, Docker Compose edge emulation, QUIC/Protobuf control plane, FastAPI/React/PostgreSQL observability.
 - **Mobility:** SUMO as primary trajectory/road-network generator; T-Drive/GeoLife for ML training; optional CARLA for presentation-only 3D clips (not on critical path).
 - **Capability-aware speculation:** Edge nodes declare workload capabilities (e.g. image/GPU); speculation manager filters predicted edges before shadow creation.
 - **Distance and ETA:** Haversine or road-network distance for region mapping and lead-time gating ($ETA \geq \text{sync_time} + \text{margin}$).
 - **Destination matrix:** Simple Markov baseline optional; GRU probability distribution is primary predictor output.
 - **Dask:** Optional parallel Python data prep for training pipelines only — does not replace Flink or Rust.
 - **Dashboard:** Leaflet/Mapbox 2D map; event timeline; shadow/sync/authority panels.
-

1. System objective

1.1 Problem statement

Mobile devices running stream-processing workloads at edge nodes incur **reactive migration latency** when they move: the new edge waits for state transfer and restoration after arrival, causing service gap, dropped continuity, and measurable handoff delay.

1.2 PREVAIL solution

PREVAIL converts edge handoff from **reactive** to **predictive**:

1. **Predict** likely next edge(s) and arrival timing from mobility history.
2. **Pre-position** application state at candidate future nodes as **warm shadows** (processing, synchronized, output suppressed).
3. **Promote** the correct shadow to **authoritative** when movement occurs, via chain-of-custody token transfer.
4. **Discard** incorrect shadows or **fallback** to reactive migration when prediction or capability constraints fail.

1.3 Research hypothesis

> If mobile edge destinations can be predicted with sufficient confidence and lead time, proactively maintaining warm state replicas at resource-feasible future edges reduces handoff latency versus reactive migration, while resource-bounded speculation limits overhead from wrong predictions.

1.4 Measurable success

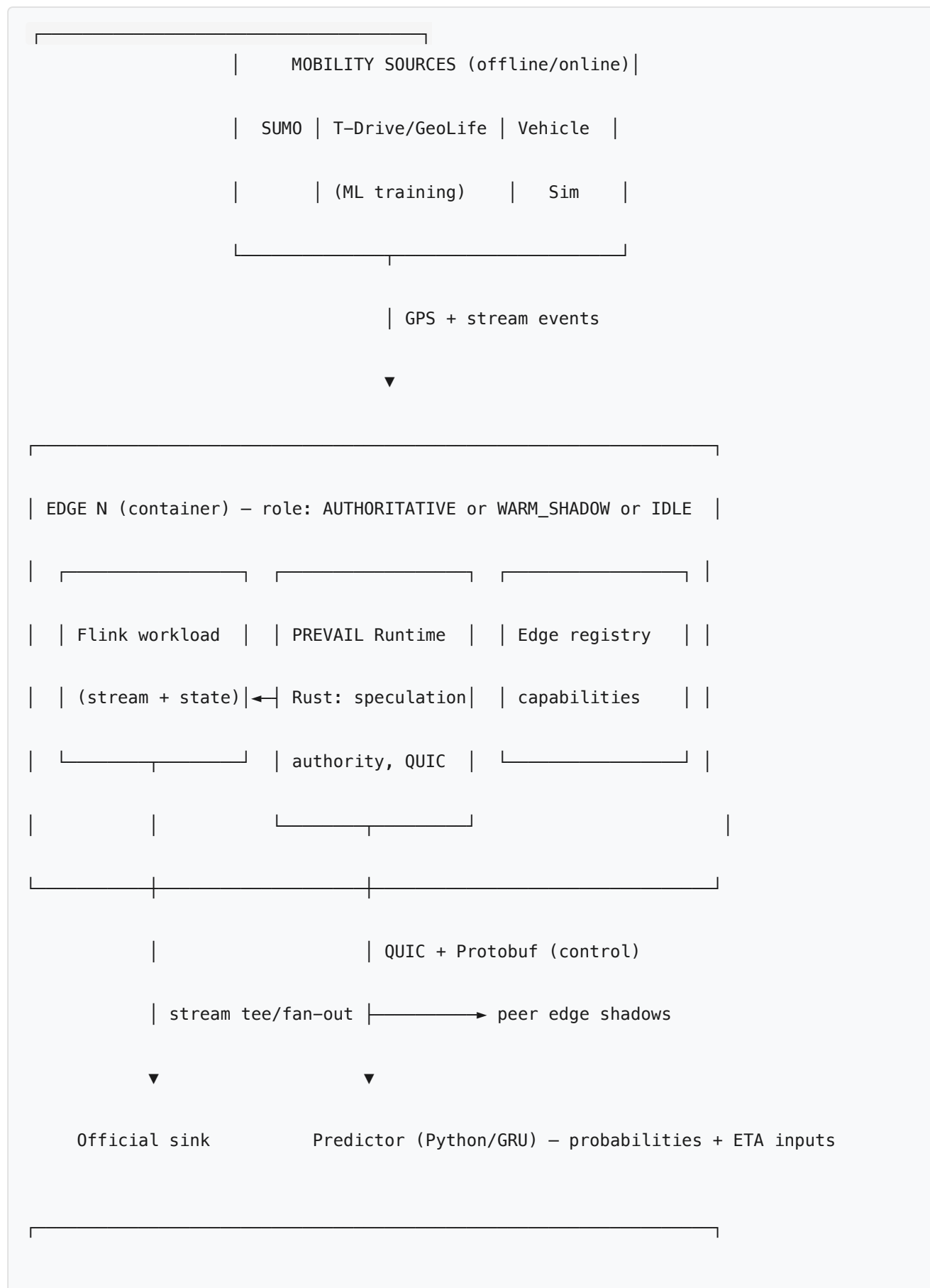
Category	Success criteria
Correctness	Exactly one authoritative publisher per session; no unsuppressed shadow output on official path
Performance	Lower p50/p95 transition latency vs baseline on correct-prediction scenarios
Research	Break-even prediction accuracy curve; overhead vs benefit under varying speculation budgets
Demo	Live dashboard shows predict → shadow → sync → promote → demote timeline

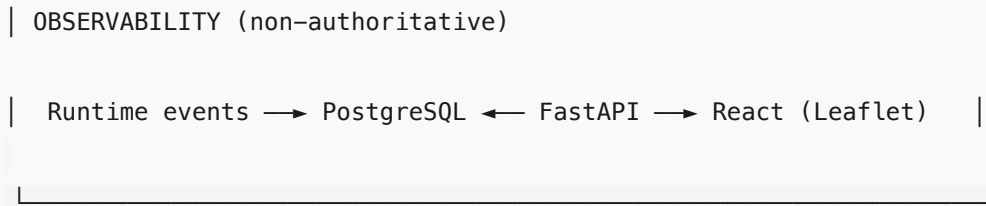
1.5 Explicit non-goals (v1 prototype)

- Physical MEC hardware deployment
 - Multi-tenant production SaaS
 - Byzantine consensus beyond epoch-based authority token
 - CARLA/SUMO as runtime dependency for handoff correctness (SUMO feeds trajectories; handoff logic is independent)
 - Dask as stream-processing engine
-

2. Finalized architecture

2.1 Logical architecture





2.2 Three planes

Plane	Responsibility	Technologies
Data	High-volume stream, state replication to shadows	Flink, app-level fan-out (multicast UDP where viable)
Control	Shadow lifecycle, promotion, authority, capability checks	Rust, QUIC, Protobuf
Observability	Metrics, timeline, topology visualization	FastAPI, WebSocket, PostgreSQL, React

Invariant: Handoff correctness never depends on PostgreSQL or the dashboard backend.

2.3 Five core subsystems (trace all work here)

1. **Trajectory Predictor** — $P(\text{next edge} \mid \text{history})$; optional Markov destination-matrix baseline.
2. **Speculation Manager** — confidence + ETA + capability + resource budget $\rightarrow$ create/skip shadows.
3. **Warm Shadow Manager** — create, sync, suppress output, discard shadows.
4. **Chain-of-Custody / Authority** — single authoritative node; epoch token; promotion/demotion.
5. **Stream Processing Layer** — Flink application state and continuous processing.

2.4 Emulated hardware model

Each Docker container = **one logical edge compute node** with configured:

- ``edge_id``, geographic region polygon(s)
- ``capabilities``: `{ stream, image, gpu, ... }`
- Resource limits (CPU/RAM) for resource-aware speculation demos

Not physical edge hardware — thesis language: **containerized distributed edge emulation**.

3. Component responsibilities

3.1 SUMO mobility generator

- Builds or imports road network; simulates vehicle route.
- Exports time-stamped GPS (and optional speed) $\rightarrow$ ****Vehicle Sim adapter**** or direct injection.
- ****Does not**** run PREVAIL logic; supplies realistic mobility input for experiments and demos.

3.2 Vehicle simulator

- Replays SUMO-exported or file-based trajectories ($A \rightarrow A \rightarrow B \rightarrow C$).

- Emits GPS/region events + application stream (GPS aggregates, sensor tuples, **image event metadata** if capability demo enabled).
- Deterministic scenario IDs for reproducible experiments.

3.3 Edge region map + distance service

- Static graph: edges as regions (polygons or cells), adjacency optional.
- GPS → `edge\_id`; distance to region boundary/centroid; **ETA** = distance / speed (with minimum floor).
- Shared by SUMO export alignment, predictor labels, speculation lead-time gate, dashboard map.

3.4 Trajectory predictor (Python, PyTorch, GRU)

- **Input:** Recent edge-id sequence, speed, direction features.
- **Output:** Probability distribution over next `edge\_id` (not a single hard label).
- **Training data:** T-Drive, GeoLife mapped to same region graph; optional **Dask** for parallel preprocessing only.
- **Export:** TorchScript/ONNX (<1 MB target); versioned artifact.
- **Optional baseline:** Destination transition matrix for comparison experiments.

3.5 Speculation manager (Rust)

- Consumes `PredictionResult`, `ETA`, local and peer resource snapshots, `SpeculationConfig`.
- **Capability filter:** Remove candidates where required workload caps absent (e.g. `needs\_image` → edge must have `supports\_image`).
- **Lead-time gate:** No shadow if $\text{ETA} < \text{estimated_sync_time} + \text{promotion_margin}$.
- **Budget:** `max\_shadows`, `min\_confidence`, cumulative cost caps.
- Emits `ShadowCreate` / `ShadowRelease` to Warm Shadow Manager.

3.6 Warm shadow manager (Rust + Flink coordination)

- Instantiates shadow Flink subtask/job on target edge container.
- Configures **output suppression** on shadow sinks.
- Drives state/stream sync per locked semantics (Phase 0 ADR).
- Reports `ShadowSyncStatus` (lag records or time-behind authoritative).
- On wrong prediction or promotion to elsewhere: **discard** shadow resources.

3.7 Chain-of-custody / authority (Rust)

- **AuthorityToken:** `{ session\_id, epoch, holder\_edge\_id, signature }`.
- Monotonic epoch on promotion; old holder demoted after `PromotionAck`.
- Crash semantics: shadow promotion only if token rules + sync threshold + authoritative timeout (documented).
- Prevents split-brain on official output path.

3.8 Flink stream processing layer

- Representative mobile workload: keyed state by `vehicle\_id` / session.
- Runs on **authoritative** edge; shadow instances mirror processing.

- Checkpointing enabled for **reactive migration baseline** and optional sync aid.
- Integration with Rust via **sidecar** (gRPC/Unix socket): location reports, authority queries, promotion callbacks.

3.9 Reactive migration baseline (comparison mode)

- No shadows: on handoff, checkpoint export → transfer → restore → resume.
- Same workload and topology; used for all PREVAIL vs traditional metrics.

3.10 Control transport

- **QUIC** + **Protobuf** between edge runtimes: prediction relay (if needed), shadow control, promotion, token transfer, heartbeats.
- Lab prototype: TLS optional; document threat model.

3.11 Stream distribution

- **Primary v1:** Application-level **fan-out** from authoritative edge to shadow QUIC/stream endpoints.
- **Optimization:** Multicast UDP where Docker network supports it; not required for correctness.

3.12 Observability stack

- **FastAPI:** REST read APIs + optional demo controls (scenario start — not authority).
- **WebSocket:** Live timeline, sync %, authority holder.
- **PostgreSQL:** Append-only experiment log, metrics samples, run metadata.
- **React + Leaflet/Mapbox:** Map, prediction panel, shadow panel, event timeline, comparison charts (Recharts).

3.13 Optional: CARLA (presentation)

- Pre-recorded or scripted 3D clip for stakeholder demos.
- **Not** on critical path for metrics or handoff correctness.

4. End-to-end execution flow

4.1 Session start

1. Vehicle sim starts scenario; session bound to `vehicle_id`.
2. GPS maps to Edge A; Edge A runtime becomes **AUTHORITATIVE**; authority token issued `epoch=0`.
3. Flink job active on A; stream ingress from sim.
4. Observability records `VehicleConnected`, `AuthorityGranted`.

4.2 Steady state on authoritative edge

1. Flink processes stream; keyed state accumulates.
2. Runtime samples trajectory; every K ticks calls predictor → `PredictionResult`.
3. Speculation manager evaluates candidates (capability, confidence, ETA, resources).

4. If approved: Warm Shadow Manager on peer Edge B creates shadow; stream tee + state sync begin.
5. Shadow reports sync progress; dashboard updates; **shadow output suppressed**.

4.3 Correct prediction handoff

1. Sim/GPS detects region change $A \rightarrow B$.
2. Runtime validates B has shadow and $\text{sync} \geq \text{threshold}$ (or policy-defined promotion rules).
3. **Promotion:** token epoch++, holder=B; A demoted; B authoritative.
4. Flink official sink on B; A shadow torn down or idle.
5. Metrics: `transition_latency_promotion` vs baseline for same trace.

4.4 Wrong prediction

1. Predicted B; vehicle enters C.
2. Discard shadow on B.
3. **Reactive migration** $A \rightarrow C$ (or C cold start if A unreachable): checkpoint path.
4. Metrics: `failed_speculation`, fallback latency, wasted shadow cost.

4.5 Capability mismatch

1. Workload requires `image`; predicted edge B lacks `supports_image`.
2. Speculation manager **never creates** shadow on B; may select capable peer or wait.
3. Fallback reactive migration to nearest capable edge in region graph.

4.6 Continuous loop

After promotion to B, predictor runs on B's trajectory context; speculation targets next edges (e.g. C); cycle repeats.

4.7 Closed-loop diagram

```
Observe → Predict → Filter(capability) → Speculate? → Create shadow → Sync  
→ Move → [Correct: Promote | Wrong: Discard + Migrate] → Observe → ...
```

5. Hardware architecture (emulated)

Logical component	Emulation	Responsibility
Edge node A–D	Docker container	Compute, Rust runtime, Flink task slot, local volumes for checkpoints
Network	Compose custom bridge	QUIC between containers; optional partition via tc/iptables in tests
Vehicle	Sim container or host process	Event generation only
Flink JobManager	Container	Job coordination (central JM + edge-labeled TMs — default ADR)
Predictor	Container or embedded lib	Inference CPU; GPU optional for training only
Observability DB	PostgreSQL container	Experiment persistence
Workstation host	Developer machine	Orchestrates Compose; not authoritative

6. Software architecture

6.1 Repository layout (target monorepo)

```
prevail/  
├─ proto/                # Protobuf schemas  
  
├─ rust/prevail-runtime/  # Edge agent  
  
├─ python/predictor/      # Train + inference  
  
├─ python/mobility/       # SUMO export adapters, region map  
  
├─ flink/                 # Job + ProcessFunction hooks  
  
├─ sim/vehicle/           # Scenario replay  
  
├─ deploy/                # docker-compose, configs  
  
├─ backend/               # FastAPI  
  
├─ frontend/              # React dashboard  
  
├─ experiments/           # Scenario YAML, runners  
  
└─ docs/                  # ADRs, this plan
```

6.2 Technology stack (final)

Layer	Choice	Role
Stream processing	Apache Flink	Application workload + state
Edge runtime	Rust	Speculation, shadows, authority, QUIC
ML	Python, PyTorch, GRU	Next-edge distribution
ML data prep (optional)	Dask	Parallel dataset preprocessing
Mobility	SUMO + T-Drive/GeoLife	Trajectories + training
Control	QUIC, Protobuf	Inter-edge messages
Stream fan-out	App tee (+ multicast optional)	Shadow input
Emulation	Docker Compose	Edge nodes
API/UI	FastAPI, React, TypeScript, Tailwind	Dashboard
Map	Leaflet or Mapbox	2D topology
Metrics store	PostgreSQL	Experiment log
3D demo (optional)	CARLA	Presentation only

7. Hardware–software interaction

1. **Host** runs `docker compose up` ; assigns each edge container CPU/RAM limits.
2. **Vehicle sim** sends UDP/TCP to **current authoritative** edge ingress port (routing policy in runtime).
3. **Edge Rust runtime** reads cgroup stats (or simulated load) for resource-aware speculation.
4. **Flink** uses container-local disk for checkpoints; baseline migration copies checkpoint blob edge-to-edge via control/data channel.
5. **SUMO** runs offline or in CI; produces trajectory files consumed by sim — no runtime coupling to Rust.
6. **Predictor** invoked over localhost HTTP or Unix socket from authoritative edge container only (decentralized inference per edge).

8. Communication and data flow

8.1 Protobuf message catalog (control plane)

Message	Direction	Purpose
`TrajectorySample`	Sim → Edge	GPS, speed, timestamp
`PredictionResult`	Predictor → Runtime	edge_id → probability
`SpeculationDecision`	Internal	Audit trail
`ShadowCreate` / `ShadowRelease`	A → B	Shadow lifecycle
`ShadowSyncStatus`	B → A	Lag/sync metric
`PromotionRequest` / `PromotionAck`	A ↔ B	Handoff
`AuthorityToken` / `AuthorityTransfer`	A → B	Custody
`DemotionNotice`	A → peers	Post-promotion cleanup
`MigrationFallbackStart/Complete`	Runtime	Wrong prediction path
`TimelineEvent`	Runtime → PG	Dashboard
`EdgeCapabilityAdvertisement`	Edge → peers	Capability-aware filtering

8.2 Data plane paths

```
Sim —stream→ Authoritative Flink ingress
Authoritative —tee→ Shadow Flink ingress(es)

Authoritative Flink → Official sink (metrics/file/Kafka lab sink)

Shadow Flink —X→ (sinks blocked)
```

8.3 Observability path

```
Runtime —async→ PostgreSQL
FastAPI ←query— PostgreSQL

FastAPI —WebSocket→ React
```

9. Security and trust model

9.1 Trust boundaries

Boundary	Trust assumption
Edge runtime ↔ edge runtime	Same lab trust domain; protobuf schema validation
Runtime ↔ Flink sidecar	Same container; local socket
Runtime ↔ Predictor	Localhost; no external trust
Dashboard ↔ FastAPI	Read-mostly; demo controls cannot forge authority token
PostgreSQL	Untrusted for correctness; append-only audit

9.2 Security properties (prototype)

- **Authority integrity:** Only token holder may enable official sink.
- **No shadow leakage:** Suppression enforced in Flink + runtime double-check.
- **Versioned proto:** Reject unknown promotion messages on critical paths.

9.3 Known weaknesses (accepted for v1)

- No mutual TLS on QUIC
- No multi-tenant isolation
- Simulated vehicle, not attested mobile device

Document for thesis; harden in future work section.

10. State management

State type	Owner	Replication
Flink keyed operator state	Authoritative Flink	Incremental sync to shadows per ADR
Authority token	Rust runtime	Transferred on promotion; not in DB
Prediction cache	Ephemeral per edge	Recomputed on trajectory
Experiment log	PostgreSQL	Durable, non-authoritative
Checkpoints	Local/MinIO (if ADR)	Baseline migration + optional shadow aid

Sync definition (must lock in Phase 0): e.g. shadow lag $\leq N$ records OR $\leq T$ ms behind authoritative keyed state.

11. Failure handling

Failure	Behavior
Wrong prediction	Discard shadow; reactive migrate to actual edge
Capability mismatch	Skip shadow; migrate to capable edge
Shadow sync incomplete at arrival	Policy: delay promotion vs promote with catch-up vs fallback migrate — **Phase 0 ADR**
Authoritative crash, warm shadow exists	Promote shadow if token + sync rules satisfied
Authoritative crash, no shadow	Session stall or recovery from checkpoint — document
Shadow crash	Recreate if time and budget allow; else migrate
Network partition	Gossip timeout; no dual authority; suppress ambiguous promotion
Predictor unavailable	Skip speculation; baseline migration only

12. Resolved conflicts and technical resolutions

Conflict	Resolution
Professor suggested Dask for parallelism	Dask **only** for optional training data prep; Flink remains stream engine
CoppeliaSim / CARLA / SUMO	**SUMO** integrated for mobility; CARLA optional demo; CoppeliaSim not used
Predict location vs predict capable server	**Two-stage:** GRU predicts edge region → capability filter on edge registry
Multicast in Docker	App fan-out required for v1; multicast optional
Central FastAPI vs decentralized PREVAIL	FastAPI observability only; authority on edge Rust
Image workload on wrong edge	Capability tags + filter before shadow; reactive fallback

13. Unresolved decisions (Phase 0 gate)

Must be ADR-documented before Phase 1 coding:

ID	Decision	Options / recommendation
ADR-01	Flink topology	**Rec:** Central JM + edge-labeled task managers
ADR-02	Shadow state sync mechanism	Checkpoint delta vs stream replay + periodic align — pick one
ADR-03	Sync metric definition	% lag vs time-behind vs checkpoint sequence
ADR-04	Rust↔Flink binding	**Rec:** Sidecar gRPC in same container
ADR-05	Promotion atomicity	Two-phase ack from old holder
ADR-06	Incomplete sync at arrival	Fallback migrate vs best-effort promote
ADR-07	Checkpoint store for baseline	Local volume vs MinIO
ADR-08	Flink version pin	Lock Java/API compatibility
ADR-09	Incomplete sync threshold default	Numeric default for experiments

14. Implementation phases

Phase 0 — Architecture lock (Week 1–2)

Build: ADRs ADR-01–09; proto v0; edge region graph; capability schema; metrics schema; failure semantics doc; demo scenario YAML.

Depends on: Nothing.

Output gate: Signed-off proto + ADRs; no implementation until closed.

Phase 1 — Emulation foundation (Week 2–4)

Build: Docker Compose; edge containers; Rust skeleton; QUIC ping; vehicle sim stub; region mapper; SUMO export pipeline (offline trajectory → sim format).

Depends on: Phase 0 proto, region graph.

Output gate: Four edges exchange Protobuf over QUIC; sim drives edge enter/exit events; SUMO trace replays on map coordinates.

Phase 2 — Flink workload (Week 4–6)

Build: Flink job; keyed state; authoritative-only sink; ingress from sim; checkpoint metrics.

Depends on: Phase 1.

Output gate: Steady processing on Edge A; checkpoint size/latency logged.

Phase 3 — Authority + reactive baseline (Week 6–8)

Build: Authority token; handoff detection; reactive migration; output gating; partition tests.

Depends on: Phase 2, ADR-05–07.

Output gate: Baseline A→B migration latencies recorded; zero dual-authority in tests.

16. Integration strategy

1. **Vertical slices:** Each phase delivers measurable milestone before next layer.
 2. **Proto-first:** All inter-edge features land in `proto/` before Rust/Flink code.
 3. **Golden scenario:** Fixed trace `scenario_golden.yaml` regression-tested from Phase 3 onward.
 4. **Feature flags:** `mode=baseline|prevail`, `speculation_enabled`, `capability_check_enabled`.
 5. **Weekly invariant tests:** Single authority; shadow suppression; promotion latency on golden trace.
-

17. Testing and validation strategy

Level	Scope
Unit	Region map, ETA, capability filter, token epoch logic
Integration	QUIC round-trip; shadow create/discard; promotion ack
System	Full Compose stack; golden + wrong-prediction scenarios
Fault	Kill authoritative/shadow; iptables partition
Experiment	E1 baseline vs PREVAIL; E2 break-even; E3 budget; E4 threshold; E5 lead-time; E6 failures
Regression	Same scenario ID → same metric within tolerance

18. Performance and reliability validation

Metrics (minimum):

- Prediction accuracy (top-1, top-k)
- Transition latency (promotion vs baseline p50/p95)
- Shadow sync time
- Bandwidth overhead (tee bytes)
- CPU/RAM overhead per shadow
- Failed speculation rate
- Break-even accuracy (derived)

Reliability: Zero tolerance for dual official output in automated tests; document recovery times for crash scenarios.

19. Technical risks and mitigations

Risk	Mitigation
Flink–Rust complexity	Sidecar boundary; thin ProcessFunction
Shadow sync too slow	Lead-time gate; tune tee; reduce state scope
Docker not true multicast	Fan-out from day one
GRU misaligned with SUMO regions	Single region graph source of truth
Scope creep (CARLA)	CARLA optional; SUMO on critical path only
pip/tooling env	Pin containers for build; Rust/Python in Docker dev images

20. Final prototype milestones

Milestone	Description
M1	Edges talk; SUMO trace replays; map shows vehicle
M2	Flink runs; baseline migration metrics
M3	Warm shadow + promotion beats baseline on golden trace
M4	GRU drives speculation; dashboard live
M5	Capability + wrong-prediction demos
M6	Experiment pack + break-even plot
M7	Single-command thesis demo

21. Traceability matrix

Requirement	Phase	Components
Predict next edge	5, 6	GRU, destination matrix baseline
Pre-position state	4	Warm Shadow Manager, Flink
Resource-bounded speculation	5	Speculation Manager
Capability-aware edges	5	Edge registry, filter
Decentralized coordination	1, 4	QUIC, edge runtime
Chain of custody	3, 4	Authority token
Flink workload	2	Flink job
SUMO mobility	1	SUMO export, sim
Distance/ETA	1, 5	Region map, ETA
Traditional baseline	3	Reactive migration
Dashboard	7	FastAPI, React, Leaflet
Research metrics	8	PostgreSQL, experiment runner

22. Appendix — Four-word architecture

PREDICT → PREPARE → PROMOTE → CONTINUE

End of PREVAIL Master Engineering Plan v2.0